

Compile-Integrated Data Pipeline (CIDP)

Compiler-Native Data Quality Scoring, Fused Deduplication, and Curriculum Scheduling Through Compiled Dataset Blocks in NeuralScript

Authors: Brandon Wiemer

Date: April 2026

Status: Research Proposal

Abstract

We propose **CIDP (Compile-Integrated Data Pipeline)**, a framework where data quality filtering, deduplication, curriculum scheduling, and loss weighting are compiled into the training binary alongside model kernels — not run as a separate Python preprocessing step. In existing workflows, data quality is addressed offline: engineers run FineWeb/DCLM classifiers, MinHash deduplication, perplexity scoring, and domain mixing on raw text before tokenization, producing a static filtered dataset. This creates a rigid pipeline where quality decisions are fixed before training begins and divorced from the model being trained. CIDP exploits NSL's compiled `dataset` blocks, first-class tokenizer, MemoryMapped data loader, and cost model to generate GPU kernels that perform quality scoring, deduplication, and curriculum ordering as part of the data loading path — fused into the training loop with zero Python overhead and with access to the model's own weights for model-aware filtering.

1. The Preprocessing Bottleneck

1.1 Current Data Quality Pipeline

A typical LLM pretraining data pipeline:

Raw crawl (100T tokens)

→ Language ID filtering (Python, CPU)	# weeks
→ Heuristic quality filtering (Python, CPU)	# days
→ Classifier-based quality filtering (GPU)	# days
→ MinHash deduplication (Python, CPU, ~1.4TB RAM)	# days-weeks
→ Tokenization (CPU)	# hours
→ Shuffling + serialization (CPU, disk I/O)	# hours
→ Static .bin file on disk	

Total preprocessing: weeks to months

Training loop:

→ Read .bin file via DataLoader (CPU→GPU transfer)
→ Forward + backward + optimizer (GPU)

Problems:

1. **Static quality decisions:** Once the dataset is filtered, decisions are frozen. If training reveals that a category of "low-quality" data actually helps (as shown by the Data-Quality Illusion paper), there's no way to include it without re-running the entire pipeline.
2. **Model-agnostic filtering:** CQF (Classifier-based Quality Filtering) trains a binary classifier on "high quality" vs "low quality" text. But quality is model-dependent — what's high quality for a code model differs from a chat model. The filter doesn't know what model will be trained on the data.
3. **CPU bottleneck:** Deduplication (MinHash) and quality scoring (fastText/DCLM classifiers) run on CPU. For 100T token datasets, this takes weeks of CPU time and ~1.4TB of RAM for the hash tables.
4. **No curriculum integration:** The filtered dataset is shuffled randomly. Curriculum learning (easy→hard ordering) requires re-running the pipeline with different quality thresholds per training phase.

1.2 NSL's Unique Position

NSL's `dataset` block is a compiled construct, not a Python iterator:

```
ns!

dataset train_data("openwebtext"):
    source = MemoryMapped("data/tokens.bin", dtype=int32)
    sequence_length = 2048
    packing = true
    pack_separator = tokenizer.eos_token_id
```

The compiler generates native code for this data pipeline. CIDP extends the `dataset` block with quality, dedup, and curriculum directives that compile into GPU kernels running alongside the training kernels.

2. Five Compiler-Native Data Quality Innovations

2.1 Innovation 1: Online Model-Aware Quality Scoring

The insight: Instead of pre-filtering with a separate classifier, use the training model itself to score data quality during training. The model's per-sample loss IS a quality signal — samples with extremely high loss are likely noise or out-of-distribution; samples with extremely low loss are already learned and provide diminishing returns.

CIDP's approach: The compiled training loop computes per-sample loss and feeds it back into the data pipeline as a quality signal:

```
ns!

dataset train_data("corpus"):
    source = MemoryMapped("data/tokens.bin", dtype=int32)
    sequence_length = 2048
    packing = true

    quality:
        mode = online                # quality scored during training, not preproc
        scoring = model_loss         # use training model's own loss as quality sig
        reweight = true              # weight loss by quality score (not hard filter)
        window = 1000                # running statistics over 1000 samples
        schedule:                    # curriculum: change quality threshold over tr
            phase1(0..50%): threshold = [0.1, 10.0] # accept wide range early
            phase2(50..90%): threshold = [0.5, 5.0]  # narrow to medium quality
            phase3(90..100%): threshold = [1.0, 3.0] # focus on hardest useful samp
```

How the compiler implements this:

Compiled Training Step with CIDP Quality Scoring:

```
1. Load batch from mmap (zero-copy)
2. Forward pass → per-sample loss (already computed for training)
3. Quality assessment (fused into loss computation):
   for each sample s in batch:
       if loss[s] < threshold_low:      # too easy – already learned
           weight[s] = 0.1              # downweight, don't discard
       elif loss[s] > threshold_high:   # too hard – likely noise
           weight[s] = 0.1              # downweight
       else:                            # useful difficulty range
           weight[s] = 1.0
4. Weighted backward: loss =  $\sum \text{weight}[s] \times \text{loss}[s] / \sum \text{weight}[s]$ 
5. Update running statistics (exponential moving average of loss distribution)
```

Integration with FASE: The fused per-layer backward-optimizer step applies the quality weights to the gradient: $\text{grad_weighted} = \text{weight}[s] \times \text{grad}[s]$. This is a single scalar multiply per sample, negligible overhead.

Why this is better than pre-filtering: The "Data-Quality Illusion" paper (Nait Saada et al., 2025) showed that CQF filtering can remove data that actually helps training. Online scoring avoids this by using the model's OWN assessment of what's useful at each training phase — data that's "noise" early may become useful later as the model improves.

2.2 Innovation 2: Compiled Tokenizer-Level Deduplication

The problem: MinHash deduplication operates on raw text at the character/word level. It requires a separate preprocessing step with massive memory (1.4TB for 100T tokens). NSL can do better because the tokenizer is a first-class compiled construct.

CIDP's approach: Deduplication operates on tokenized sequences during the MemoryMapped data loading phase, using the tokenizer's BPE vocabulary as a natural hash domain:

```
ns!

dataset train_data("corpus"):
    source = MemoryMapped("data/tokens.bin", dtype=int32)
    dedup:
        method = minhash_online      # deduplicate during loading, not preprocessing
        n_hashes = 128               # MinHash signature size
        bands = 16                   # LSH bands
        threshold = 0.8              # Jaccard similarity threshold
        scope = batch                # dedup within batch (or epoch, or global)
```

Compiled dedup kernel:

Compiled DataLoader with Online Dedup:

```
1. mmap read: load next N sequences from disk (zero-copy)
2. For each sequence s (GPU kernel, parallelized):
    # Compute MinHash signature from token n-grams
    for hash_fn in 0..128:
        min_hash[hash_fn] = MIN over all token 5-grams:
            hash_fn(token_ngram)

    # LSH banding: check signature against Bloom filter
    for band in 0..16:
        band_hash = hash(min_hash[band*8 .. (band+1)*8])
        if bloom_filter.contains(band_hash):
            # Potential duplicate – mark for soft dedup (downweight, not remove)
            weight[s] *= 0.1
        bloom_filter.insert(band_hash)

3. Emit weighted batch to training loop
```

Key advantages over offline dedup:

- **GPU-accelerated:** The MinHash computation runs on the GPU alongside training, using idle SMs during data loading. No CPU bottleneck.
- **Bloom filter in GPU global memory:** The dedup state is a compact Bloom filter (~100MB for 10B token dataset) in GPU global memory, not a 1.4TB hash table in CPU RAM.
- **Soft dedup:** Instead of removing duplicates (losing information), CIDP downweights them via the quality weight system. This matches SoftDedup's finding that soft deduplication achieves the same perplexity with 26% fewer steps while preserving accuracy.
- **Online/streaming:** Works on MemoryMapped data without a separate preprocessing pass. The Bloom filter accumulates over the training epoch.

2.3 Innovation 3: Compile-Time Data Mixing Optimization

The problem: Multi-source pretraining (web + code + books + wiki) requires choosing mixing ratios. Current approaches use manual tuning (LLaMA's 67% web / 4.5% code / etc.) or expensive grid search (DCLM). The "Data Mixing Laws" paper (ICLR 2025) showed that optimal ratios can be predicted from small-scale experiments, but this still requires running multiple training runs.

CIDP's approach: The compiler analytically evaluates mixing ratios using the cost model and weight analysis:

ns!

```
dataset train_data:
  sources:
    web = MemoryMapped("data/web_tokens.bin", quality_signal="dclm_score")
    code = MemoryMapped("data/code_tokens.bin", quality_signal="none")
    books = MemoryMapped("data/books_tokens.bin", quality_signal="none")
    wiki = MemoryMapped("data/wiki_tokens.bin", quality_signal="none")

  mixing:
    mode = auto # compiler optimizes mixing ratios
    objective = downstream_proxy # optimize for downstream task proxy
    constraint = { web >= 0.5 } # at least 50% web data
```

When `--weights` is provided, the compiler computes per-source perplexity using the model's current weights. Sources where the model has low perplexity (already learned) get downweighted; sources where the model has high perplexity (still learning) get upweighted. This implements the perplexity-correlation data selection approach (Mizrahi et al., ICLR 2025) as a compile-time pass.

The mixing ratios become compile-time constants in the DataLoader's sampling kernel, with optional phase-based adjustments:

```
Compiled DataLoader with Optimized Mixing:
Phase 1 (0-80% of training): web=0.60, code=0.20, books=0.10, wiki=0.10
Phase 2 (80-95%): web=0.40, code=0.30, books=0.15, wiki=0.15
Phase 3 (95-100%): web=0.30, code=0.35, books=0.20, wiki=0.15
```

2.4 Innovation 4: Fused Curriculum Scheduling

The problem: Curriculum learning (ordering data from easy→hard) improves convergence but is undermined by learning rate decay — the best data arrives when the LR is lowest (Curriculum-LR paper, Nov 2025). Current implementations require pre-computing difficulty scores and pre-sorting the dataset.

CIDP's approach: The compiler generates a curriculum scheduler that operates on the quality scores computed online (Innovation 1), synchronized with the LR schedule:

```

nsl

dataset train_data("corpus"):
    ...
    curriculum:
        mode = ascending                # easy → hard
        difficulty = model_loss         # use training loss as difficulty proxy
        lr_aware = true                 # counteract LR decay on high-quality data
        buffer_size = 10000             # maintain a priority queue of 10K samples

```

How it works:

Compiled Curriculum Scheduler:

Maintain a priority queue of `buffer\_size` samples in GPU memory.

Every N steps:

1. Load N new samples from mmap
2. Score each sample: `difficulty = forward_pass_loss(sample)`
3. Insert into priority queue (sorted by difficulty)
4. For next N training steps, draw from priority queue:
 - Phase 1: draw from easy end (low difficulty)
 - Phase 2: draw from medium
 - Phase 3: draw from hard end

LR-aware adjustment:

When LR enters decay phase, the curriculum scheduler:

- Increases weight of hard samples (they need higher effective LR)
- Applies LR compensation: `effective_lr[sample] = lr × (difficulty / mean_difficulty)`
- This is equivalent to per-sample gradient scaling, which FASE handles natively

Integration with FASE: Per-sample gradient scaling is implemented as a scalar weight in the FASE fused backward-optimizer step. No additional kernel launches.

2.5 Innovation 5: Compile-Time Data Validation

The problem: Training data can contain format errors, encoding issues, truncated sequences, or adversarial content that causes training instability (loss spikes). These are typically caught by offline validators, but some only manifest at the tokenized level.

CIDP's approach: The compiler generates validation checks fused into the DataLoader kernel:

```

nsl

dataset train_data("corpus"):
    ...
    validate:
        min_unique_tokens = 0.1          # at least 10% unique tokens (catch repetitive
        max_special_ratio = 0.05         # at most 5% special/control tokens
        max_consecutive_repeat = 10      # no single token repeated >10 times
        entropy_floor = 2.0              # minimum token entropy per sequence
        nan_check = true                  # verify no NaN/inf in embeddings

```

These checks are compiled into the DataLoader's GPU kernel:

```

Compiled Validation Kernel (per sequence):
unique_count = count_unique(tokens)
if unique_count / seq_len < 0.1:
    weight = 0.0 # skip this sequence

special_count = count(tokens where token < special_token_boundary)
if special_count / seq_len > 0.05:
    weight = 0.0

max_run = max_consecutive_run(tokens)
if max_run > 10:
    weight = 0.0

entropy = -Σ p(token) × log(p(token))
if entropy < 2.0:
    weight *= 0.1 # downweight low-entropy sequences

```

These checks add ~2μs per sequence on the GPU — negligible compared to the forward pass. The validation runs on the GPU in the same kernel as the mmap read and dedup check, with zero CPU involvement.

3. The Compiled Data Pipeline Architecture

Standard Python pipeline:

CPU: [raw text] → [filter] → [tokenize] → [dedup] → [shuffle] → [.bin file]

GPU: [read .bin] → [pad/pack] → [forward] → [loss] → [backward]

Boundary: CPU↔GPU transfer for every batch (pinned memory, async copy)

Data quality: offline, static, model-agnostic

CIDP compiled pipeline:

GPU: [mmap read] → [validate+dedup+quality] → [pack] → [forward] → [weighted loss] → [backward]

Everything is one compiled binary. No CPU involvement in the data path.

Data quality: online, dynamic, model-aware

3.1 Integration with Existing NSL Infrastructure

NSL Subsystem	CIDP Usage
MemoryMapped (DataLoader)	Zero-copy mmap provides raw tokenized data
SequencePacker	PCA packing operates on validated, deduped sequences
Tokenizer (BPE)	Token vocabulary used as hash domain for MinHash
Cost Model (M37)	Per-sample compute cost informs curriculum difficulty
FASE	Per-sample loss weighting fused into backward
WGGO	Data mixing ratios as ILP variables
Fused Linear CE (CSHA)	Quality weights applied in loss kernel

3.2 WGGO Integration

CIDP adds data pipeline decisions to the WGGO ILP:

New WGG0 variables:

```
mix[source] ∈ [0, 1]           # mixing ratio per data source
dedup_scope ∈ {none, batch, epoch, global} # dedup granularity
curriculum_phase[step_range] ∈ {easy, medium, hard} # difficulty targeting
quality_threshold[phase] ∈ [0, 10] # loss-based quality window
```

New constraints:

```
 $\sum \text{mix}[\text{source}] = 1.0$ 
dedup memory ≤ available GPU global memory - training memory
curriculum buffer ≤ activation memory budget
```

4. NSL Language Surface

ns!

```
# Full CIDP-enabled dataset block
dataset train_data:
    sources:
        web = MemoryMapped("data/web_tokens.bin")
        code = MemoryMapped("data/code_tokens.bin")

    sequence_length = 2048
    packing = true
    pack_separator = tokenizer.eos_token_id

    mixing: auto                # compiler optimizes ratios

    quality:
        mode = online
        scoring = model_loss
        reweight = true
        schedule:
            warmup(0..10%): threshold = [0.0, 100.0] # accept everything initially
            main(10..90%):  threshold = [0.5, 8.0]   # filter extremes
            anneal(90..100%): threshold = [1.0, 4.0] # focus on hard-but-useful

    dedup:
        method = minhash_online
        n_hashes = 128
        threshold = 0.8
        soft = true                # downweight don't remove

    curriculum:
        mode = ascending
        difficulty = model_loss
        lr_aware = true
        buffer_size = 10000

    validate:
        min_unique_tokens = 0.1
        max_special_ratio = 0.05
        entropy_floor = 2.0

# Training uses the CIDP-enabled dataset
train(model=m, epochs=1):
    optimizer: AdamW(lr=3e-4)
    data: source = train_data, batch_size = 32
    step(batch):
        # batch.weights are computed by CIDP (quality × dedup × curriculum)
        let logits ← m.forward_train(batch.input_ids, true)
```

```
let logit = m.forward_train(batch.input_ids, true)
let loss = weighted_cross_entropy(logits, batch.labels, batch.weights)
```

bash

```
$ nsl build --cidp-report --weights model.safetensors model.nsl
```

```
=== CIDP Data Pipeline Report ===
```

```
Sources: web (80%), code (20%) → optimized: web (62%), code (38%)
```

```
Reason: model perplexity on code is 2.3× higher than web → needs more code
```

```
Quality scoring: online model-loss, reweighted (not filtered)
```

```
Estimated useful fraction: 78% of web, 91% of code
```

```
Low-quality downweighted: ~22% of web data (repetitive/boilerplate)
```

```
Deduplication: MinHash (128 hashes, 16 bands, threshold=0.8)
```

```
Bloom filter: 64 MB in GPU global memory
```

```
Estimated duplicate rate: ~15% (soft-deduped, not removed)
```

```
Effective training speedup from dedup: ~12%
```

```
Curriculum: ascending difficulty, LR-aware
```

```
Buffer: 10K samples × 2048 tokens × 4 bytes = 80 MB
```

```
Phase transitions: step 10K→50K→90K
```

```
Validation: 4 checks, ~2µs/sequence overhead
```

```
Estimated rejection rate: 3% of samples (set to weight=0)
```

```
Data pipeline overhead: 0.8% of total training step time
```

```
(all checks run on GPU, fused into DataLoader kernel)
```

5. Why PyTorch Cannot Do This

Capability	PyTorch/HuggingFace	CIDP
Quality scoring	Offline classifier (CPU, days)	Online model-loss (GPU, zero extra cost)
Deduplication	Offline MinHash (CPU, 1.4TB RAM)	Online GPU Bloom filter (64MB GPU memory)
Data mixing	Manual ratios or grid search	Compile-time optimization from weight perplexity
Curriculum	Pre-sort dataset offline	Dynamic priority queue with LR-aware scheduling
Validation	Separate preprocessing script	Fused into DataLoader GPU kernel ($\sim 2\mu\text{s}/\text{seq}$)
Loss weighting	Custom training loop changes	Compiled into FASE fused backward
Data pipeline location	CPU (Python DataLoader, GIL-bound)	GPU (compiled kernel, zero Python)

6. Implementation Plan

Component	Location	LOC (est.)	Description
Quality scorer	<code>ns1-codegen/src/cidp_quality.rs</code>	400	Online loss-based quality scoring codegen
MinHash dedup	<code>ns1-codegen/src/cidp_dedup.rs</code>	500	GPU MinHash + Bloom filter kernel gen
Data mixer	<code>ns1-codegen/src/cidp_mixer.rs</code>	300	Compile-time mixing ratio optimization
Curriculum scheduler	<code>ns1-codegen/src/cidp_curriculum.rs</code>	400	Priority queue + LR-aware scheduling
Validation kernel	<code>ns1-codegen/src/cidp_validate.rs</code>	250	Fused validation checks
Dataset block extension	<code>ns1-ast/src/block.rs</code>	150	AST nodes for quality/dedup/curriculum
WGGO integration	<code>ns1-codegen/src/wggo.rs</code>	100	Data mixing as ILP variables

Total: ~2,100 LOC. Dependencies: `dataloader.rs`, `memory_mapped.rs`, `tokenizer.rs`, `fused_linear_ce.rs`, `fase.rs`, `wggo.rs`

7. Conclusion

CIDP completes the tenth and final paper in the NSL compiler-native optimization suite:

#	Paper	Domain
1	WRGA	PEFT / Adapters
2	CSHA	Attention Fusion
3	CFTP (FASE+PCA)	Gradient Accumulation + Packing
4	CEP	Pruning + NAS
5	CPDT	Distributed Training
6	WGGO	Global ILP Solver
7	CFIE	Inference Optimization
8	CPKD	Knowledge Distillation
9	CIDP	Data Quality + Filtering

CIDP's central insight: **data quality is not a preprocessing problem — it's a compilation problem**. The compiler knows the model architecture, the target hardware, the optimizer configuration, and (optionally) the pre-trained weights. This knowledge enables quality decisions that offline preprocessors cannot make: model-aware scoring, LR-synchronized curriculum, hardware-optimal dedup data structures, and loss-integrated quality weighting. The entire data pipeline runs on the GPU as part of the compiled training binary, with zero Python overhead and zero CPU↔GPU serialization for data quality decisions.

References

- Nait Saada, T., et al. "The Data-Quality Illusion: Rethinking Classifier-Based Quality Filtering." arXiv 2510.00866, 2025.
- Li, J., et al. "DataComp-LM: In Search of the Next Generation of Training Sets." NeurIPS 2024.
- He, X., et al. "SoftDedup: Efficient Data Reweighting for LLM Pre-training." arXiv 2024.
- Mizrahi, D., et al. "Language Models Improve When Pretraining Data Matches Target Tasks." ICLR 2025.
- Penedo, G., et al. "FineWeb: Decanting the Web for the Finest Text Data." 2024.
- Wettig, A., et al. "Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance." ICLR 2025.